SAE 5.02

Cahier des charges initial

Nom du projet : CryptEasy

Membres : Diogo CARVALHO, Sam KAFAI, Yanis DISCOURS, Yani SAI

BUT R&T, parcours Cybersécurité des systèmes industriels et des infrastructures - IUT Nancy-Brabois

15 septembre 2026

1. Note d'intention

Échanger des fichiers sensibles de manière sécurisée reste aujourd'hui compliqué pour un utilisateur non technique : les outils existants (GPG en ligne de commande, par exemple) demandent de comprendre des notions de cryptographie asymétrique avant de pouvoir simplement chiffrer un fichier. À l'inverse, les solutions plus simples d'accès reposent parfois sur des implémentations peu robustes, donnant une impression de sécurité qui ne correspond pas à la réalité.

L'objectif de ce projet est de développer un outil de chiffrement et déchiffrement de fichiers doté d'une interface graphique, pensé pour être accessible à un utilisateur non expert (génération de clés et chiffrement en quelques clics, réglages par défaut sûrs) tout en restant utile à un utilisateur plus avancé, qui pourra ajuster certains paramètres (algorithme, taille de clé). La sécurité réelle de l'outil est un objectif central du projet : il s'appuiera exclusivement sur des bibliothèques cryptographiques reconnues et maintenues, sans réimplémentation des primitives cryptographiques.

2. Description du livrable

Le livrable est une application écrite en Python, avec une interface graphique, permettant de :

- Générer une paire de clés (RSA ou courbe elliptique, au choix)
- Chiffrer un fichier pour un destinataire, via un chiffrement hybride (AES pour le fichier, clé publique du destinataire pour protéger la clé AES)
- Déchiffrer un fichier reçu à l'aide de sa clé privée (elle-même protégée par mot de passe)
- Gérer un carnet de contacts : enregistrer les clés publiques des destinataires habituels pour éviter de les réimporter à chaque envoi
- (Fonctionnalité complémentaire, selon l'avancement) Signer un fichier et vérifier une signature reçue

L'application sera packagée sous la forme d'un exécutable autonome (via PyInstaller), afin que l'utilisateur final puisse chiffrer et déchiffrer ses fichiers directement, sans avoir à installer Python ni aucune dépendance.

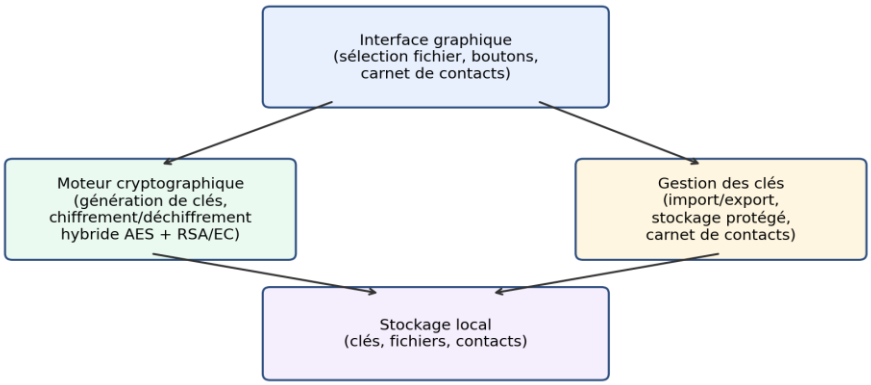


Figure 1 - Architecture logicielle envisagée

3. Planning prévisionnel

Le projet se déroule sur environ 10 semaines, de la validation du sujet jusqu'à la soutenance. Le planning ci-dessous sera comparé au déroulement réel dans le rapport final.

Phase	Semaines	Contenu	Responsable(s)
Cahier des charges	S1	Rédaction et validation du sujet avec l'enseignant	Tous
Conception	S2-S3	Architecture logicielle, choix des bibliothèques, maquette de l'interface	Tous
Développement - moteur cryptographique	S4-S6	Génération de clés, chiffrement/déchiffrement hybride AES + RSA/EC	Membre 1
Développement - interface graphique	S4-S6	Interface web (Streamlit ou Flask) : sélection de fichiers, écrans de génération/gestion des clés	Membre 2
Développement - gestion des clés	S6-S7	Import/export, stockage protégé, carnet de contacts	Membre 3
Packaging	S6-S8	Génération d'un exécutable autonome (PyInstaller) intégrant l'ensemble de l'application	Membre 4
Intégration	S7-S8	Assemblage des modules, correction des incohérences	Tous
Tests	S8	Tests fonctionnels et cas limites (fichiers volumineux, mauvais mot de passe, etc.)	Tous
Documentation & rapport	S9	Rédaction de la documentation et du rapport final	Tous
Préparation soutenance	S10	Support de présentation, répétition, démonstration	Tous

4. Risques identifiés et parades envisagées

Risque	Impact	Parade envisagée
Mauvaise répartition du temps entre les 4 membres	Retard sur le planning	Points d'avancement hebdomadaires courts, tâches assignées clairement dès le départ
Écart de compétences en cryptographie entre les membres	Ralentissement sur le module cœur	Travail en binôme sur les parties sensibles, documentation interne partagée
Dérive du périmètre (ajout de fonctionnalités en cours de route)	Non-respect des délais	Définition d'un périmètre minimal (MVP) figé après le cahier des charges
Erreur dans la gestion des clés (perte, mauvaise manipulation)	Perte de données de test, blocage en démonstration	Tests systématiques, utilisation de données factices pour les démonstrations
Prise en main tardive de la bibliothèque graphique choisie	Retard sur le développement de l'interface	Prototype d'interface simple dès la phase de conception, avant le développement complet

5. Outils envisagés

- Langage : Python 3
- Bibliothèque cryptographique : cryptography (Python), pas de réimplémentation des algorithmes
- Interface : Streamlit ou Flask (choix à trancher en phase de conception)
- Packaging : PyInstaller, pour générer un exécutable autonome
- Gestion de version et de projet : GitHub (dépôt commun ; l'enseignant sera invité en tant que collaborateur)
- Communication : Discord
- Tests : pytest
- Rédaction du rapport : Word